

https://github.com/sushi2dragon/MRM/blob/main/project_3_code.ipynb

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

[REDACTED]

np.random.seed(69)

[REDACTED]

df= pd.read_csv("Country-data.csv")

[REDACTED]

country= df["country"].tolist()
X= df.drop("country", axis=1)

[REDACTED]

X = (X-X.mean()) / X.std(ddof=0)

[REDACTED]

X_corr = (1/167)* X.T.dot(X)

[REDACTED]

[REDACTED]

eig_values, eig_vectors = np.linalg.eig(X_corr)

[REDACTED]

total_eig = np.sum(eig_values)

[REDACTED]

print("sum of eigenvalues:", np.sum(eig_values))

[REDACTED]

explained_variance = (eig_values / np.sum(eig_values)) * 100
```

```

print("Explained Variance for PC1:", explained_variance[0])
print("Explained Variance for PC2:", explained_variance[1])
plt.figure(figsize=(8, 4))
plt.bar(range(X.shape[1]), explained_variance, alpha=0.6)
plt.ylabel("% of explained variance")
plt.xlabel("Dimensions")
plt.show()

print("Explained Variance Dimensions:", explained_variance.shape)
print("Number of Dimensions in Data (X):", X.shape[1])

pc1 = X.dot(eig_vectors[:, 0])
pc2 = X.dot(eig_vectors[:, 1])

X_pca = np.vstack((pc1, pc2)).T

```

```

def plot_scatter(pc1, pc2):
    fig, ax = plt.subplots(figsize=(15, 8))

    for i, spec in enumerate(country):
        plt.scatter(pc1[i], pc2[i], label=spec, s=20, color="black")
        ax.annotate(str(i + 1), (pc1[i], pc2[i]))

    ax.set_xlabel('PC1', fontsize=15)
    ax.set_ylabel('PC2', fontsize=15)
    ax.axhline(y=0, color="grey", linestyle="--")
    ax.axvline(x=0, color="grey", linestyle="--")

    plt.grid()
    plt.axis([-4, 4, -3, 3])
    plt.show()

plot_scatter(pc1, pc2)

```

```

def initialize_centroids(X, k):
    indices = np.random.choice(X.shape[0], k, replace=False)
    return X[indices, :]

```

```
def assign_to_clusters(X, centroids):
    distances = np.linalg.norm(X[:, np.newaxis, :] - centroids, axis=2)
    return np.argmin(distances, axis=1)

def update_centroids(X, labels, k):
    centroids = np.zeros((k, X.shape[1]))
    for i in range(k):
        cluster_points = X[labels == i, :]
        centroids[i, :] = np.mean(cluster_points, axis=0)
    return centroids
```

```
def kmeans(X, k, max_iters=100):
    centroids = initialize_centroids(X, k)
    for _ in range(max_iters):
        labels = assign_to_clusters(X, centroids)
        new_centroids = update_centroids(X, labels, k)
        # Check for convergence
        if np.all(centroids == new_centroids):
            break
        centroids = new_centroids
    return labels, centroids
```

```
k = 3
```

```
labels, centroids = kmeans(X_pca, k)
```

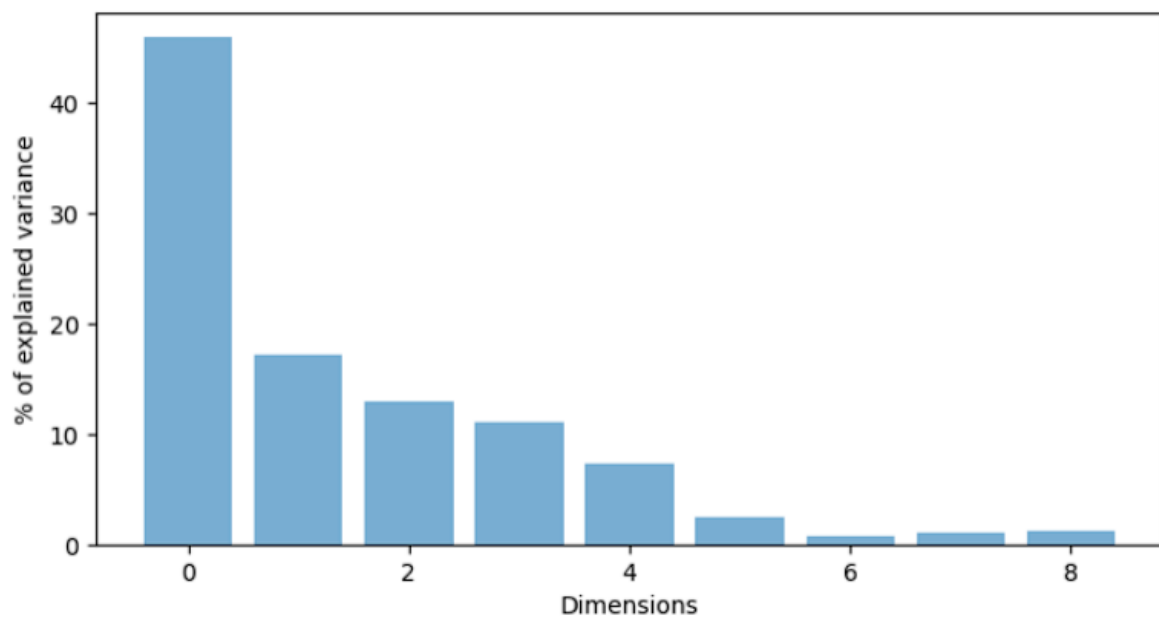
```
plt.figure(figsize=(12, 8))
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=labels, cmap='viridis',
            edgecolors='k', s=50)
plt.scatter(centroids[:, 0], centroids[:, 1], c='red', marker='X', s=75,
            label='Cluster Centroids')
plt.title('K-means')
plt.xlabel('PC1')
```

```
plt.ylabel('PC2')  
plt.legend()  
plt.grid(True)  
plt.show()
```

Analysing the outputs:

For this dataset, after calculating, we get the sum of the eigenvalues as 9

After plotting the scree plot, we can see the two most important pcs,

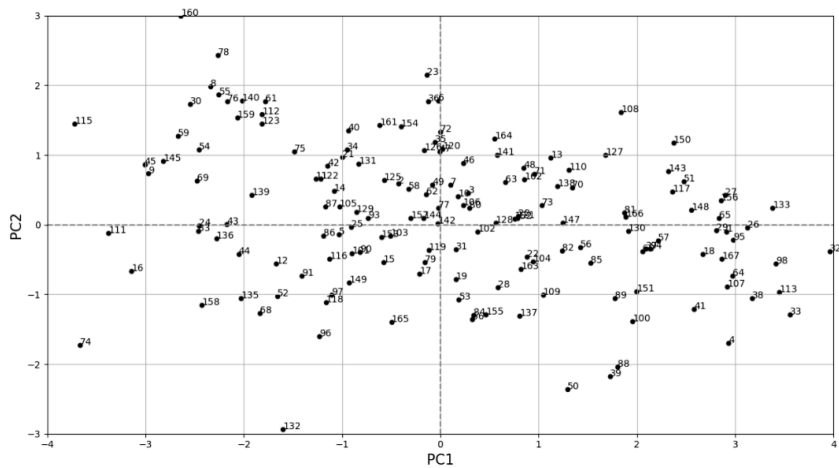


Pc1 with explained variance of 47%

Pc2 with explained variance of 17%

These are the two principal axes we will use

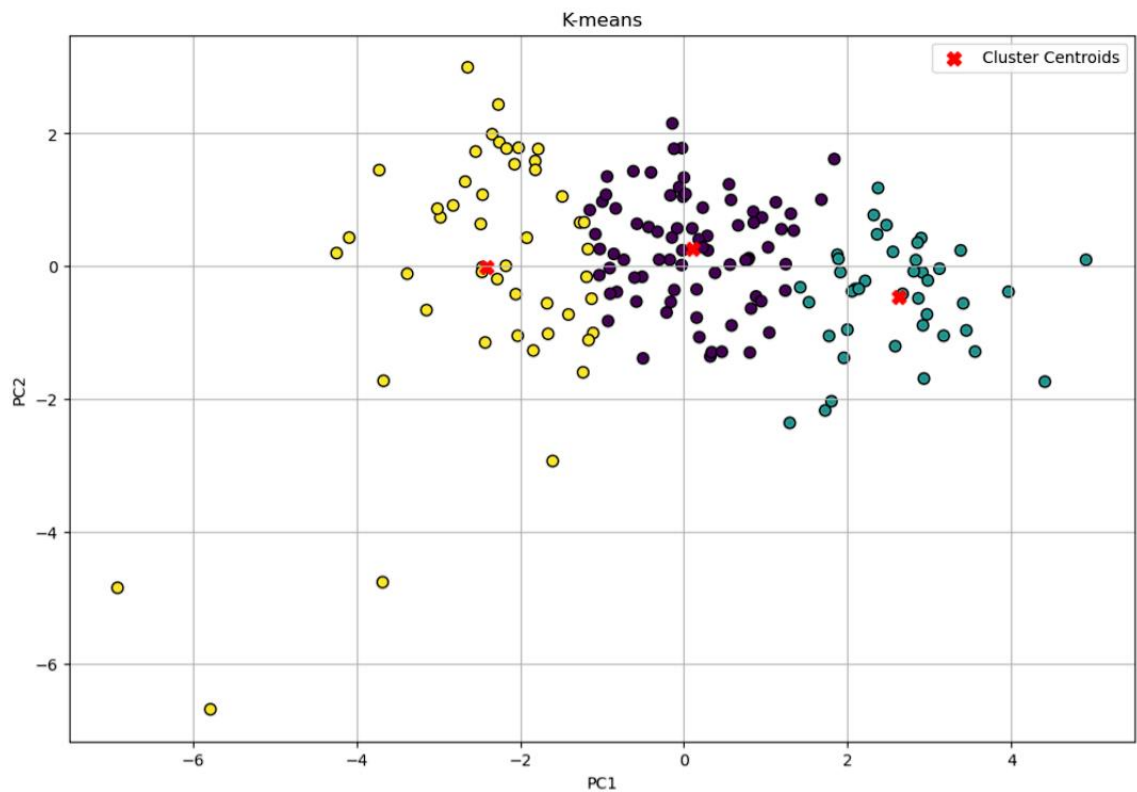
The Pca graph with 63.13% of the original dataset's variance.



From the graph, there aren't any directly visible distinct clusters of datapoints, a few minor outliers may indicate anomalies

The PCA graph doesn't really give strong evidence for distinct clusters coming up

Kmeans graph:



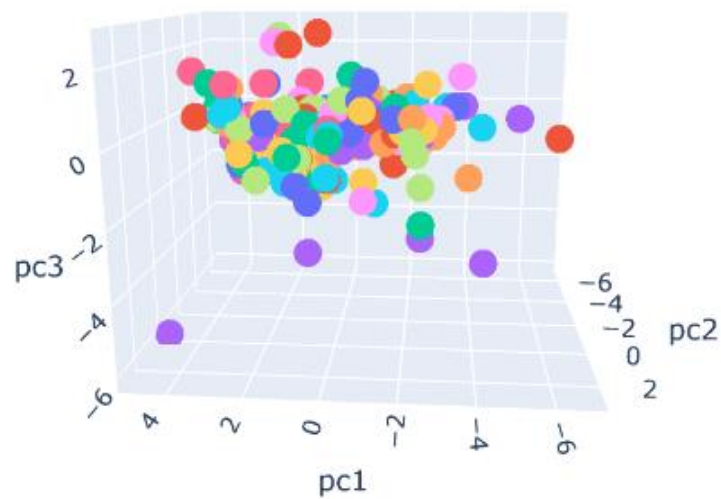
The clusters are rather close to each other which shows that there aren't many natural clusters in the data

I tried plotting the data in 3d to get a better understanding of it

Once with mplot 3d

Other time with plotly because its better because it lets you move around and freely explore the graph

Pca plot in 3d



Kmeans plot in 3d

K-means 3D

